



**Universidad Nacional Autónoma
de México.**

Facultad de Ingeniería



“Sistemas Operativos”

Semestre: 2026-2

Grupo de Teoría: 07

Profesor:

DR. GUNNAR EYAL WOLF ISZAEVICH

– Documento Escrito –

“(Micro) Sistema de Archivos Multihilos”

Fecha de entrega: 21/05/2026

Equipo:

Adrián Axel Arzate Ríos

David Díaz Antunez

Índice

1	Introducción	3
2	Objetivo	4
3	Desarrollo	5
3.1	Archivo <code>main.py</code>	5
3.2	Archivo <code>fiunamfs.py</code>	5
3.3	Archivo <code>worker.py</code>	6
3.4	Pruebas realizadas	6
4	Conclusiones	8

1. Introducción

En este proyecto se desarrolló un micro sistema de archivos multihilos compatible con FiUnamFS, un sistema de archivos simple utilizado con fines académicos para comprender cómo se organiza la información dentro de un dispositivo de almacenamiento.

En lugar de trabajar directamente sobre un disco físico, el sistema utiliza una imagen de disco llamada `fiunamfs.img`. Esta imagen simula un dispositivo de almacenamiento y contiene la estructura interna del sistema de archivos, incluyendo un superbloque, un directorio plano y una zona de datos donde se guardan los archivos.

La implementación se realizó en Python utilizando FUSE, una herramienta que permite montar sistemas de archivos en espacio de usuario. Gracias a esto, la imagen `fiunamfs.img` puede montarse como si fuera una carpeta normal del sistema operativo. Una vez montada, es posible interactuar con ella mediante comandos comunes de Linux como `ls`, `cp`, `cat` y `rm`.

Este proyecto integra dos temas importantes de la materia: sistemas de archivos y administración de procesos. Por una parte, se trabaja con la lectura, escritura y eliminación de archivos dentro de una imagen binaria. Por otra parte, se incorpora el uso de hilos y mecanismos de sincronización para controlar el acceso concurrente a la imagen del sistema de archivos.

2. Objetivo

El objetivo principal del proyecto es implementar un micro sistema de archivos compatible con **FiUnamFS**, utilizando Python y FUSE para permitir que una imagen de disco pueda montarse como un directorio dentro del sistema operativo.

Los objetivos específicos son:

1. Leer e interpretar correctamente la estructura interna de una imagen `fiunamfs.img`.
2. Validar que la imagen corresponda al sistema de archivos **FiUnamFS**.
3. Listar los archivos almacenados dentro del sistema de archivos.
4. Copiar archivos desde **FiUnamFS** hacia el sistema operativo anfitrión.
5. Copiar archivos desde el sistema operativo anfitrión hacia **FiUnamFS**.
6. Eliminar archivos almacenados dentro de **FiUnamFS**.
7. Implementar operaciones concurrentes mediante hilos.
8. Utilizar mecanismos de sincronización para evitar inconsistencias al acceder o modificar la imagen.
9. Probar el sistema usando comandos comunes de Linux.

3. Desarrollo

El sistema de archivos proporcionado por el profesor, **FiUnamFS**, se encuentra almacenado dentro de una imagen binaria llamada `fiunamfs.img`. Esta imagen representa un disco simulado y contiene toda la información necesaria para manejar los archivos.

La estructura general de **FiUnamFS** se divide en tres partes principales:

1. **Superbloque:** contiene la información general del sistema de archivos, como la firma **FiUnamFS**, la versión, la etiqueta del volumen, el tamaño de los clústeres, el número de clústeres del directorio y el número total de clústeres.
2. **Directorio:** es un directorio plano, es decir, no maneja subdirectorios. Cada entrada del directorio guarda información sobre un archivo, como su nombre, tamaño, clúster inicial, fecha de creación y fecha de modificación.
3. **Zona de datos:** es el espacio donde se almacena el contenido real de los archivos.

El sistema utiliza asignación contigua, por lo que cada archivo debe guardarse en una secuencia continua de clústeres. Por esta razón, al copiar un archivo nuevo hacia **FiUnamFS**, el programa debe buscar un bloque de espacio libre suficientemente grande para almacenarlo.

3.1. Archivo `main.py`

El archivo `main.py` funciona como punto de entrada del programa. Este archivo recibe dos argumentos desde la terminal: el punto de montaje y la imagen del sistema de archivos.

El punto de montaje es el directorio vacío donde se montará **FiUnamFS**, mientras que la imagen es el archivo `fiunamfs.img` que contiene el sistema de archivos.

Un ejemplo de ejecución es:

```
mkdir -p mnt
python main.py mnt ../fiunamfs.img
```

Desde este archivo se inicializa FUSE y se crea el objeto principal encargado de manejar las operaciones del sistema.

3.2. Archivo `fiunamfs.py`

El archivo `fiunamfs.py` contiene la implementación principal del sistema de archivos. En este archivo se definen las operaciones que FUSE utiliza para interactuar con la imagen.

Entre sus funciones principales se encuentran:

- Leer y validar el superbloque.

- Interpretar las entradas del directorio.
- Listar archivos.
- Leer archivos.
- Escribir archivos.
- Crear nuevas entradas.
- Eliminar archivos existentes.

Una de las funciones más importantes es la encargada de montar el sistema de archivos. Esta función lee los primeros bytes de la imagen para obtener la información del superbloque y validar que realmente se trate de un sistema **FiUnamFS**. Después se analiza el directorio para identificar los archivos existentes y construir una representación en memoria.

Otra parte importante es la construcción del mapa de espacio libre. Este mapa permite saber qué clústeres están ocupados y cuáles están disponibles. Primero se marcan como ocupados los clústeres reservados para el superbloque y el directorio. Después se revisan los archivos existentes y se marcan los clústeres que utiliza cada uno. Este proceso es necesario para poder copiar nuevos archivos hacia la imagen sin sobrescribir información existente.

3.3. Archivo `worker.py`

El archivo `worker.py` contiene la lógica relacionada con el manejo de hilos. En este archivo se implementa un hilo trabajador encargado de realizar operaciones de lectura y escritura sobre la imagen.

Para coordinar las operaciones se utiliza una cola de tareas y mecanismos de sincronización. Esto permite que las solicitudes al sistema de archivos se procesen de forma controlada, evitando que varias operaciones modifiquen la imagen al mismo tiempo.

El uso de sincronización es importante porque la imagen `fiunamfs.img` es un recurso compartido. Si dos operaciones intentaran escribir al mismo tiempo, podrían producirse inconsistencias o corrupción de datos. Por esta razón, el sistema separa las tareas de entrada y salida y las procesa de manera ordenada.

3.4. Pruebas realizadas

El proyecto fue probado en un entorno Linux. Para la ejecución se utilizó un entorno virtual de Python y se instalaron las dependencias necesarias, incluyendo FUSE y `fusepy`.

El flujo básico de ejecución fue el siguiente:

```
mkdir -p mnt
python main.py mnt ../fiunamfs.img
```

Una vez montado el sistema, se realizaron pruebas desde otra terminal. Primero se listaron los archivos dentro de `mnt`:

```
ls -la mnt
```

El sistema mostró archivos como:

```
logo.png  
mensaje.jpg  
README.org
```

Después se probó copiar un archivo desde FiUnamFS hacia Linux:

```
cp mnt/README.org README_copiado.org
```

También se probó copiar un archivo desde Linux hacia FiUnamFS:

```
echo "hola_desde_linux" > prueba.txt  
cp prueba.txt mnt/prueba.txt  
cat mnt/prueba.txt
```

Finalmente, se probó eliminar un archivo dentro de FiUnamFS:

```
rm mnt/prueba.txt
```

Estas pruebas comprobaron que el sistema puede listar archivos, leer archivos, copiar archivos hacia la imagen y eliminar archivos dentro del sistema montado.

También se consideraron algunos casos extremos, como intentar leer un archivo inexistente, copiar un archivo con nombre demasiado largo, copiar directorios o intentar escribir archivos cuando no hay suficiente espacio. Estos casos son importantes porque permiten evaluar la estabilidad del sistema y verificar que los errores se manejen correctamente.

4. Conclusiones

El desarrollo de este proyecto permitió comprender de manera práctica cómo se organiza internamente un sistema de archivos. A diferencia del uso cotidiano de archivos y carpetas desde el sistema operativo, en este proyecto fue necesario trabajar directamente con una imagen binaria, interpretar su superbloque, recorrer su directorio y ubicar el contenido de los archivos dentro de la zona de datos.

El uso de FUSE facilitó la interacción con **FiUnamFS**, ya que permitió montar la imagen como si fuera un directorio normal del sistema. Gracias a esto, operaciones comunes como **ls**, **cp**, **cat** y **rm** pudieron utilizarse para probar el funcionamiento del sistema de archivos.

También se reforzó la importancia de la sincronización en operaciones concurrentes. Al trabajar con una imagen compartida, es necesario evitar que dos operaciones modifiquen al mismo tiempo la estructura del sistema de archivos, ya que esto podría provocar inconsistencias o corrupción de datos.

Una parte interesante del desarrollo se relaciona con la exposición realizada sobre GPU y sistemas operativos. En ambos casos se observa que no todos los componentes de software funcionan de la misma forma en todos los sistemas operativos. Así como en el tema de GPU existen dependencias, controladores y capas de compatibilidad específicas para cada entorno, en este proyecto se observó que herramientas como FUSE están pensadas principalmente para sistemas tipo UNIX/Linux.

Esto muestra la importancia de considerar la naturaleza del sistema operativo al desarrollar software de bajo nivel. La compatibilidad no depende únicamente del lenguaje de programación, sino también de las bibliotecas disponibles, el tipo de sistema de archivos, las llamadas al sistema y la forma en que cada sistema operativo maneja recursos como archivos, permisos y dispositivos.

En conclusión, el proyecto permitió integrar temas de sistemas de archivos, procesos, concurrencia y compatibilidad entre sistemas operativos. Además, ayudó a comprender que el diseño de software cercano al sistema depende fuertemente del entorno donde se ejecuta.